

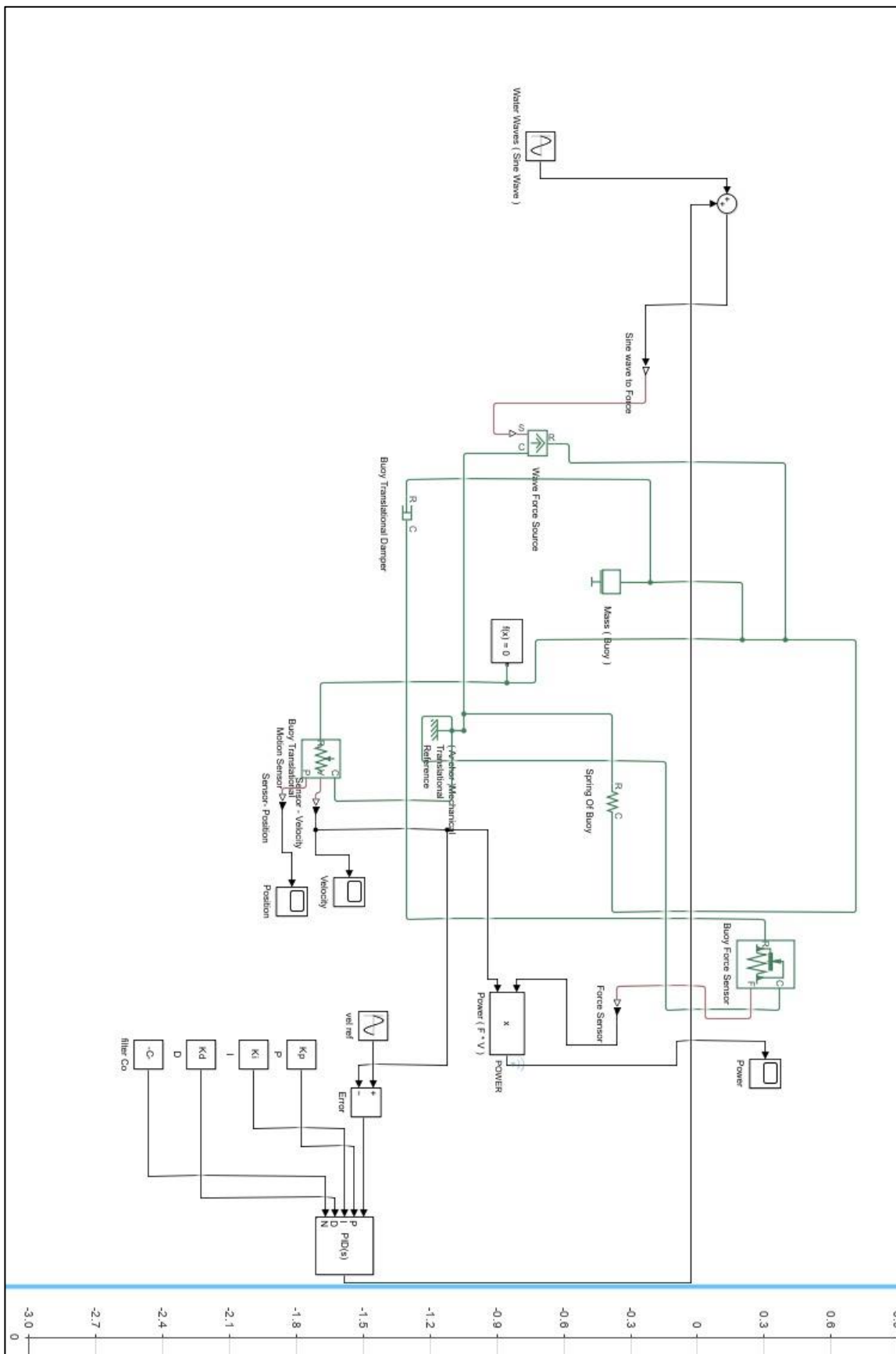


Fig - Simulink model of WEC buoy & PID controller configuration

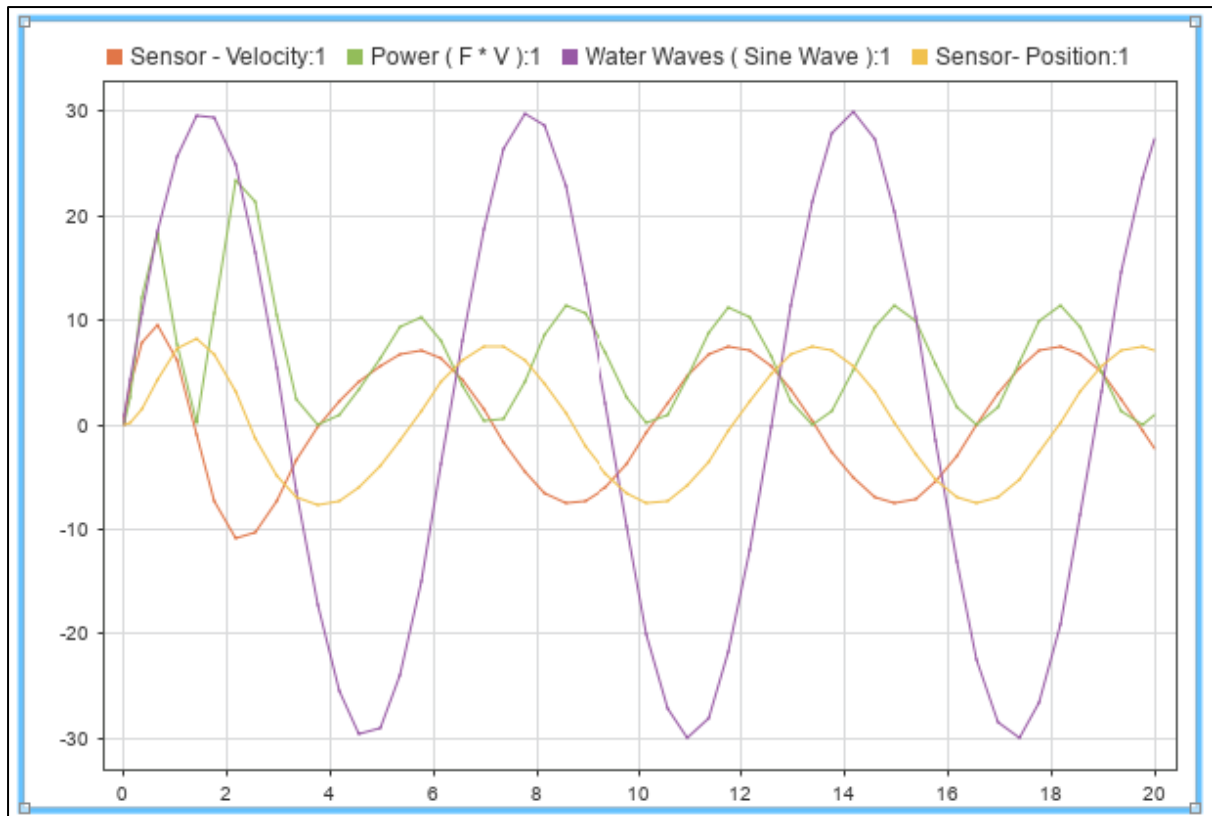


Fig - Simulation results (Power, Velocity , Position) in initial PID conditions , $K_p = 1$, $K_i = 0$, $K_d = 0$.

Block Parameters: PID Controller1

PID 1dof (mask) (link)

This block implements continuous- and discrete-time PID control algorithms and includes advanced features such as anti-windup, external reset, and signal tracking. You can tune the PID gains automatically using the 'Tune...' button (requires Simulink Control Design).

Controller: PID Form: Parallel

Time domain:

☒ Continuous-time

☐ Discrete-time

Discrete-time settings

Sample time (-1 for inherited): -1

Compensator formula

$$P + I \frac{1}{s} + D - \frac{N}{1 + N \frac{1}{s}}$$

Main Initialization Saturation Data Types State Attributes

Controller parameters

Source: Internal

Proportional (P): 41.7883891640729

Integral (I): 326.14861098262 ☐ Use I*Ts (optimal for codegen)

Derivative (D): -0.221579532776993

Filter coefficient (N): 70.1237917224969 ☒ Use filtered derivative

Automated tuning

Select tuning method: Transfer Function Based (PID Tuner App) Tune...

☒ Enable zero-crossing detection

OK Cancel Help Apply

Fig - PID Parameters when auto tuned using Transfer Function Based (PID Tuner App) in simulink for first order transfer function.

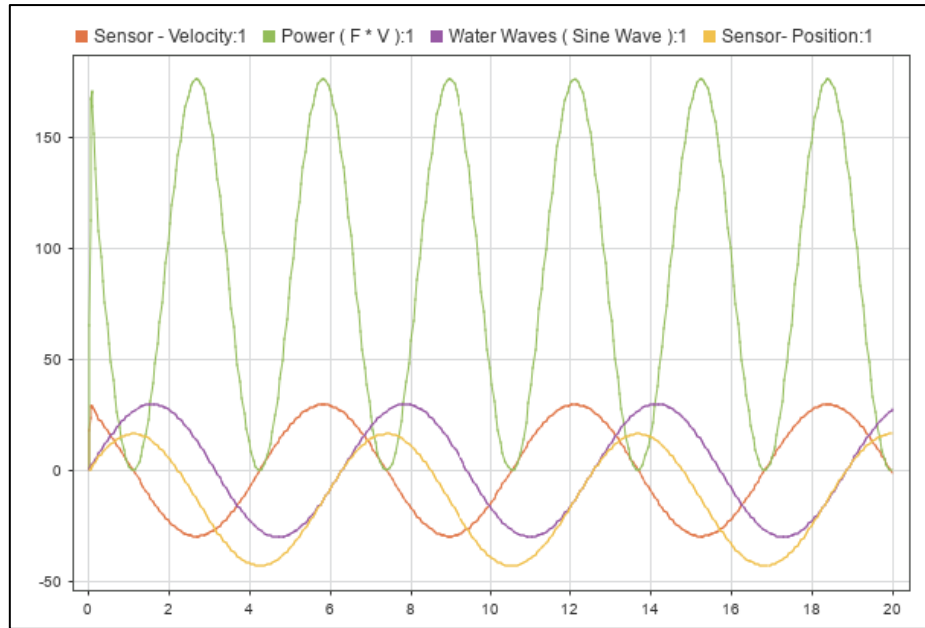


Fig - Simulation results (Power, Velocity , Position) in auto tuned PID conditions , $K_p = 41.788$, $K_i = 326.148$, $K_d = -0.2215$.

Block Parameters: PID Controller1

PID 1dof (mask) (link)

This block implements continuous- and discrete-time PID control algorithms and includes advanced features such as anti-windup, external reset, and signal tracking. You can tune the PID gains automatically using the 'Tune...' button (requires Simulink Control Design).

Controller: PID Form: Parallel

Time domain:

☒ Continuous-time

☐ Discrete-time

Discrete-time settings

Sample time (-1 for inherited): -1

Compensator formula

$$P + I \frac{1}{s} + D \frac{N}{1 + N \frac{1}{s}}$$

Main Initialization Saturation Data Types State Attributes

Controller parameters

Source: internal

Proportional (P): 28

Integral (I): 0 ☐ Use I*Ts (optimal for codegen)

Derivative (D): 12

Filter coefficient (N): 423.582423418783 ☒ Use filtered derivative

Automated tuning

OK Cancel Help Apply

Fig - PID Parameters when auto tuned using ML modelling in simulink for first order transfer function.

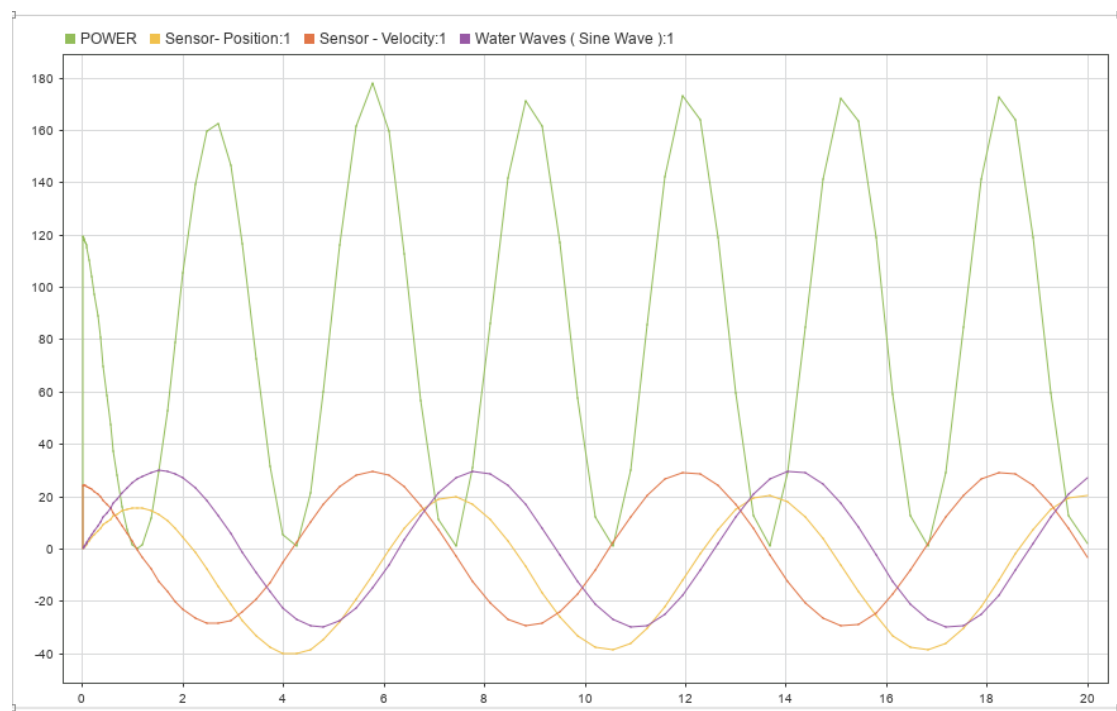


Fig -

**Simulation results (Power, Velocity , Position) in ML tuned PID conditions ,
 $K_p = 28$, $K_i = 0$, $K_d = 12$.**

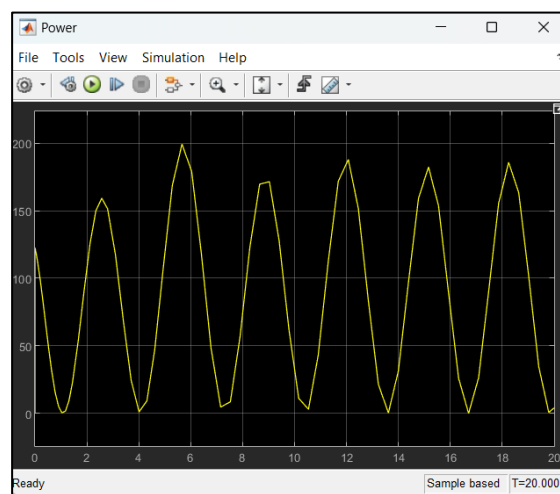
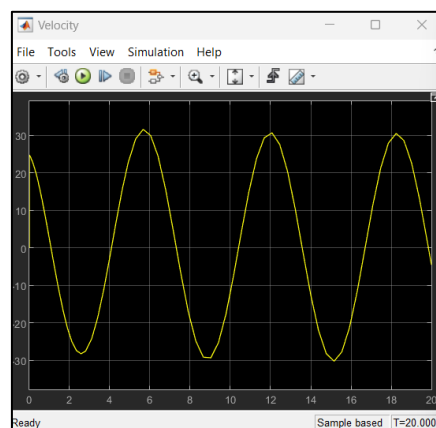
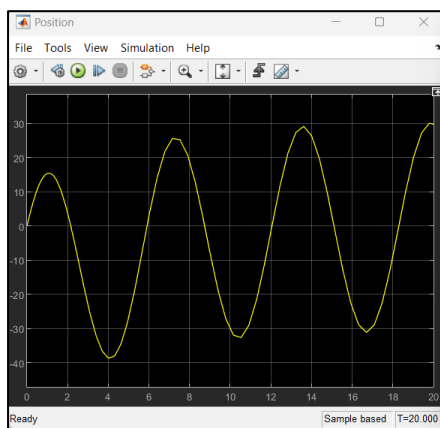


Fig- Individual plots of position , velocity and power at ML tuned PID parameters

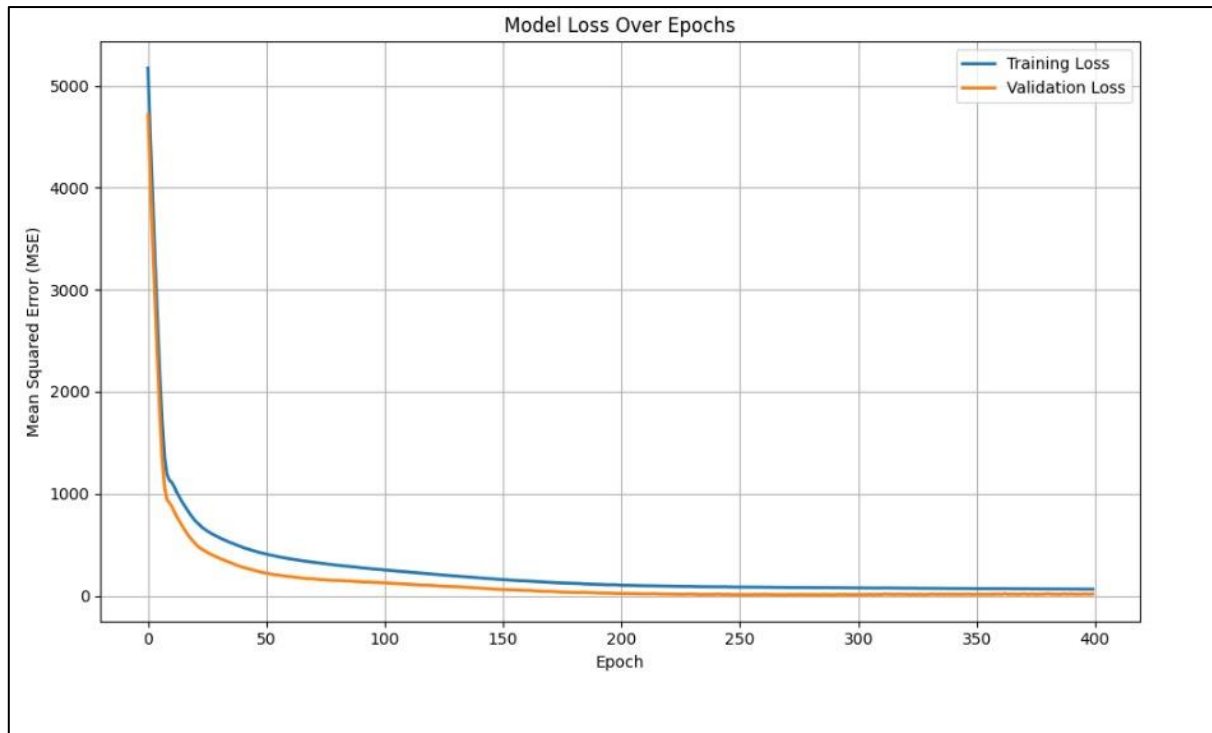


Fig - Loss curves show a steady convergence of training and validation errors.

```

1  % Define PID gain ranges
2  kp_range = linspace(1, 50, 10);
3  ki_range = linspace(0, 10, 5);
4  kd_range = linspace(0, 10, 5);
5
6  % Initialize dataset
7  dataset = [];
8
9  % Loop over all combinations
10 for kp = kp_range
11     for ki = ki_range
12         for kd = kd_range
13             % Assign PID values to base workspace
14             assignin('base', 'Kp', kp);
15             assignin('base', 'Ki', ki);
16             assignin('base', 'Kd', kd);
17
18             % Simulate the model
19             simOut = sim('DL_Project', ... % <-- change if model name is different
20                 'StopTime', '20', ...
21                 'SaveOutput', 'on', ...
22                 'SaveFormat', 'Dataset', ...
23                 'ReturnWorkspaceOutputs', 'on');
24
25             % Extract 'logsout' data
26             logsout = simOut.get('logsout');
27             power_data = [];
28
29             % Search for 'POWER' signal
30             for i = 1 : logsout.numElements
31                 signal = logsout.get(i);
32                 if strcmp(signal.Name, 'POWER')

```

```

        power_data = signal.Values.Data;
        break;
    end
end

% Compute average power
if ~isempty(power_data)
    avg_power = mean(power_data);
else
    warning('POWER signal not found for Kp=%.2f, Ki=%.2f, Kd=%.2f. Using NaN.', kp, ki, kd);
    avg_power = NaN;
end

% Append to dataset
dataset = [dataset; kp, ki, kd, avg_power];
end
end

% Remove NaN rows
dataset_clean = dataset(~isnan(dataset(:, 4)), :);

% Write to CSV
csvwrite('pid_power_dataset.csv', dataset_clean);

disp('✅ CSV file "pid_power_dataset.csv" generated successfully.');
```

Fig -MATLAB Code for pid_power_dataset creation